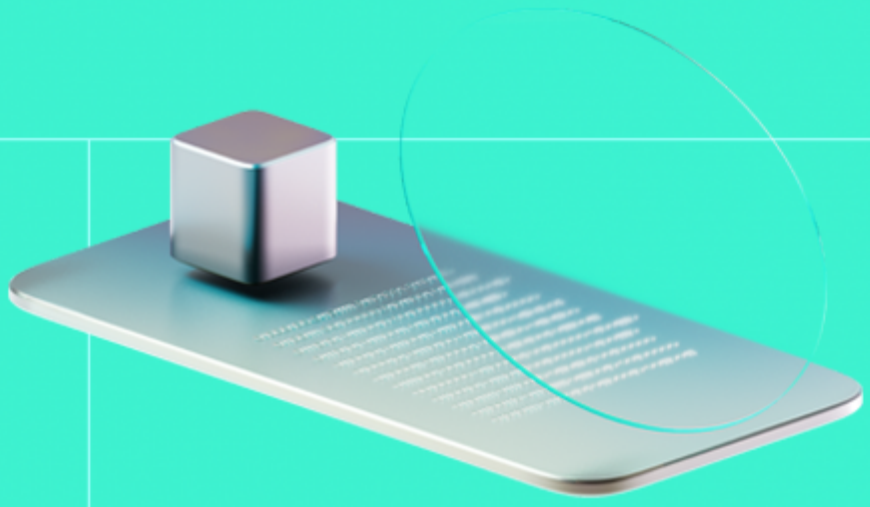
HACKEN

Smart Contract Code Review And Security Analysis Report



Customer: Overlayer

Date: 12/03/2026

We express our gratitude to the Overlayer team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

The Overlayer protocol is built on top of Ethereum to enhance existing stablecoins by minting overlaid assets backed by Aave. It embeds yield generation, censorship resistance, and trustless access directly into the asset design.

Document

Name	Smart Contract Code Review and Security Analysis Report for Overlayer
Audited By	Khrystyna Tkachuk, Kerem Solmaz
Approved By	Ivan Bondar
Website	https://overlayer.fi/
Changelog	27/02/2026 - Preliminary Report
	12/03/2026 - Final Report
Platform	Ethereum, Base
Language	Solidity
Tags	ERC20, ERC4626, Yield Farming, Staking, Vault, Permit Token
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

Repository	https://github.com/Overlayerfi/contracts/tree/main
Initial Commit	471a134
Final Commit	519c9e9

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

11	11	0	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	2
Low	2

Vulnerability	Severity	Status
F-2026-15235 - Users Can Renounce BLACKLISTED_ROLE and Bypass Administrative Restrictions	High	Fixed
F-2026-15250 - Hub Chain OverLayer Supply Reduction After OFT Transfers Lead to supply() DoS	High	Fixed
F-2026-15229 - Cooldown Timer Reset on Repeated Calls Leads to Extended Staking on Previously Queued Assets	Medium	Fixed
F-2026-15242 - Default Admin Can Assign Blacklisted Role Without Enforcing Blacklist Activation Constraints	Medium	Fixed
F-2026-15236 - Instant Change Of Backing Contract Address Can Disable All Vault Operations	Low	Fixed
F-2026-15237 - Emergency Disable Function Only Halts Minting Despite NatSpec Claiming Both Mint And Redeem Are Disabled	Low	Fixed
F-2026-15233 - Floating Pragma Across All Contracts	Info	Fixed
F-2026-15239 - No Cancellation Mechanism For Pending Per-Block Redeem Limit Proposals	Info	Fixed
F-2026-15244 - Lack of Event Emitting for Key State Changes	Info	Fixed
F-2026-15245 - Inconsistency Between Documentation and Implementation in removeCollateralManagerRole()	Info	Fixed
F-2026-15246 - Redundant Redeem Rate-Limit Check Implemented Both As Modifier And Inline Logic	Info	Fixed

Documentation quality

- NatSpec comments are comprehensive across all public and external functions.
- Technical description is provided via auto-generated solidity-docgen output and a README with mint/redeem/stake/compound flow overview.
- A formal protocol specification document (whitepaper or spec) is not provided.

Code quality

- The project makes extensive use of OpenZeppelin v5.0.2 contracts (ERC20, ERC4626, AccessControl, ReentrancyGuard, SafeERC20, Pausable) and LayerZero OFT for cross-chain functionality.
- The development environment is properly configured with multi-compiler Hardhat setup, Prettier formatting, and Solhint linting.

Test coverage

Code coverage of the project is 50% (branch coverage).

- Core user flows (mint, redeem, staking, cooldown, Aave supply/compound), negative cases, and multi-user scenarios are covered.
- Administrative lifecycle functions and the OverlayerBacking proxy have limited coverage.

Table of Contents

System Overview

Overlayer is a collateral-backed stablecoin protocol built on EVM-compatible chains. The system centers on **OverlayerWrap**, an ERC20 stablecoin with mint and redeem against configured collateral (and its Aave interest-bearing aToken). Minting and redeeming are rate-limited per block where minting can be paused; blacklisting is supported with a time-delayed activation (15 days). The protocol uses **LayerZero OFT** for cross-chain transfers to a designated hub chain.

Collateral is managed through a **time-delayed collateral spender** pattern: a new spender is proposed by a collateral manager role and must be accepted by the proposed contract after a 10-day interval, limiting single-point-of-failure risk. The **OverlayerBacking** contract acts as the backing manager entry point, inheriting **AaveHandler** to supply and withdraw collateral on Aave v3, compound yield, and distribute rewards between the staking vault and a protocol rewards dispatcher.

Staking is implemented via **StakedOverlayerWrap**, an ERC4626 vault that wraps OverlayerWrap. It supports a **cooldown mechanism**: when enabled, users request unstake and wait a configurable cooldown (up to 90 days) before withdrawing. During cooldown, shares are burned and the underlying tokens are held in **OverlayerWrapSilo**, which only the staking vault can withdraw from. **StakedOverlayerWrapCore** adds blacklisting (with time-delayed activation), redistribution of blacklisted balances, and reward distribution; it triggers backing `compound()` on mint to harvest yield before new share issuance.

The **StakedOverlayerWrap** vault does not implement explicit “rewards per share” or epoch-based accounting. Instead, yield is distributed implicitly through share price appreciation, consistent with the ERC4626 vault model, where rewards are reflected in `totalAssets()`. Deposits and withdrawals occur at the current share price, and deposit/withdraw operations trigger `compound()` before minting or burning shares. This ensures that accrued yield is realized and reflected in the vault state prior to share supply changes, preserving proportional ownership across participants.

Access control is implemented through **SingleAdminAccessControl**, a custom pattern on top of OpenZeppelin AccessControl with a single admin, two-step admin transfer (propose + accept), and restrictions on granting or renouncing the admin role directly.

Files in scope

- **OverlayerWrap.sol** – Primary stablecoin token contract. Implements ERC20 functionality and exposes mint and redeem entry points. Includes blacklisting functionality with time-delayed activation.
- **OverlayerWrapCore.sol** – Implements the core mint and redeem logic. Enforces per-block rate limiting (maximum mint/redeem per block with optional whitelist bypass), pause controls, collateral spender validation, and LayerZero OFT integration. Minting and redemption are restricted to the designated hub chain.
- **CollateralSpenderManager.sol** – Manages the collateral spender role with a time-delay mechanism. A new collateral spender must be proposed and can accept the role only after a 10-day delay. Only the proposed address may finalize acceptance. Exposes the currently approved collateral spender for collateral transfers.
- **OverlayerWrapCollateral.sol** – Stores collateral configuration parameters, including main collateral and corresponding aToken addresses and decimals. Serves as a base contract for

collateral-aware components. Uses `SingleAdminAccessControl` for administrative initialization and management.

- **StakedOverlayerWrap.sol** – ERC4626-compliant staking vault for OverlayerWrap with an optional cooldown mechanism. When cooldown is enabled, users must request unstaking and wait for the cooldown period before claiming assets from the silo. May trigger backing `compound()`. Manages cooldown duration and silo integration.
- **StakedOverlayerWrapCore.sol** – Implements core staking functionality. Includes ERC4626 logic, blacklisting with time-delayed activation, redistribution of blacklisted balances, and reward distribution via `transferInRewards()`.
- **OverlayerWrapSilo.sol** – Escrow contract used during the cooldown period. Holds OverlayerWrap tokens between unstake request and claim. Only the staking vault is authorized to withdraw funds.
- **OverlayerBacking.sol** – Entry point for backing allocation management. Inherits `AaveHandler`. Accepts the collateral spender role from OverlayerWrap and provides asset recovery functionality (excluding protocol-designated collateral).
- **AaveHandler.sol** – Integrates with Aave v3 for collateral supply and withdrawal. Implements yield compounding by minting OverlayerWrap from surplus aToken balances and distributing rewards between the staking vault and dispatcher. Supports time-delayed updates to the Aave pool and rewards allocation parameters. `supply()` is callable by OverlayerWrap; `compound()` is permissionless.
- **SingleAdminAccessControl.sol** – Custom access control module with a single default admin. Implements two-step admin transfer (propose and accept). Supports role grant, revoke, and renounce operations, with additional protection preventing external grant or renounce of the default admin role.
- **OverlayerWrapCoreTypes.sol** – Defines shared data structures used across the system, including:
  - `Order` (benefactor, beneficiary, collateral, amounts) for mint and redeem operations.
  - `StableCoin` (address, decimals) for collateral configuration.

Privileged roles

OverlayerWrap / OverlayerWrapCore:

- `DEFAULT_ADMIN_ROLE` - Grant/revoke all other roles; can call `setMaxMintPerBlock`, `executeMaxRedeemPerBlockChange`, `proposeMaxRedeemPerBlock`, `whitelistMaxRedeemPerBlockUser`, `rescueNative`; remove collateral managers (`removeCollateralManagerRole`).
- `GATEKEEPER_ROLE` – can disable mint and pause/unpause supply backing functionality
- `COLLATERAL_MANAGER_ROLE` – can propose new collateral spender (10-day delay); can call `approveCollateral` (approve token allowances for current spender).
- `CONTROLLER_ROLE` – can `disableAccount` / `enableAccount` enforsing blacklisting for the account.
- `BLACKLISTED_ROLE` – accounts with this role cannot mint, redeem, or transfer (treated as disabled).

CollateralSpenderManager:

- `COLLATERAL_MANAGER_ROLE` – can propose new collateral spender.

StakedOverlayerWrapCore:

- `DEFAULT_ADMIN_ROLE` – can manage `OverlayerWrapBacking` address.
- `REWARDER_ROLE` – can transfer OverlayerWrap rewards into the vault via `transferInRewards()`.

- `CONTROLLER_ROLE` – set blacklisted roles (when blacklist is active); can enable blacklist and redistribution by calling appropriately `setBlackListTime` / `setRedistributionTime`.
- `STAKE_RESTRICTED_ROLE` – account with this role cannot stake or receive shares via deposit/mint.
- `WHOLE_RESTRICTED_ROLE` – account with this role cannot transfer, stake, unstake, or withdraw; balance can be redistributed by admin via `redistributeLockedAmount`.

StakedOverlayerWrap:

- `DEFAULT_ADMIN_ROLE` – can call `setCooldownDuration()`, `setWithdrawAaveDuringCompound()`

OverlayerWrapSilo:

- `_STAKING_VAULT` – Only the staking vault address (immutable) can call withdraw.

OverlayerBacking:

- Owner – accept collateral spender, recover non-protocol assets.

AaveHandler:

- Owner – admin withdraw, propose/accept Aave and dispatcher allocation, approvals; onlyProtocol for supply (OverlayerWrap only).
- **OverlayerWrap** (protocol) – only OverlayerWrap can call `supply()` (`onlyProtocol`).

Potential Risks

- **Scope Definition and Security Guarantees**: The audit does not cover all code in the repository. Contracts outside the audit scope may introduce vulnerabilities, potentially impacting the overall security due to the interconnected nature of smart contracts.
- **Dependency on External Logic for Implemented Logic**: The implemented `AaveHandler` logic depends on external contract `OvaDispatcher` not covered by the audit. This reliance introduces risks if these external contracts are compromised or contain vulnerabilities, affecting the audited project's integrity.
- **Interactions with External DeFi Protocols**: Dependence on external DeFi protocols (Aave, LayerZero) inherits their risks and vulnerabilities. This might lead to direct financial losses if these protocols are exploited, indirectly affecting the audited project.
- **Single Points of Failure and Control**: Parts of the system rely on a limited set of privileged roles and operational processes. All privileged functions are secured through a **Gnosis Safe** multi-signature wallet. While this setup simplifies coordination and introduces additional protective mechanisms, it also means that overall availability and decision execution may depend on a small number of entities, increasing sensitivity to operational issues or targeted compromise.
- **Aave Liquidity Constraints May Delay Direct Redemption of Collateral Token:** Redemption into the underlying stablecoin (USDC/USDT) depends on the liquidity conditions of the respective Aave pool. During periods of high utilization or temporary market stress, immediate withdrawal of the base asset from Aave may be limited. However, users are still able to redeem their position from Overlayer by receiving the corresponding **Aave aToken** on a 1:1 basis. In such cases, the limitation only affects the instant withdrawal of base liquidity from Aave, not the ability to exit the Overlayer protocol or maintain exposure to the underlying interest-bearing position.

Findings

Vulnerability Details

F-2026-15235 - Users Can Renounce BLACKLISTED_ROLE and Bypass Administrative Restrictions - High

Description:

The **OverlayerWrap** stablecoin contract implements a `BLACKLISTED_ROLE` mechanism to restrict specific accounts from transferring tokens. This restriction is enforced using OpenZeppelin's **AccessControl** library.

```
function disableAccount(
    address account_
) external blacklistAllowed onlyRole(CONTROLLER_ROLE) {
    _grantRole(BLACKLISTED_ROLE, account_);
    emit DisableAccount(account_);
}
```

The **OverlayerWrap** contract inherits from **SingleAdminAccessControl**, which itself utilizes OpenZeppelin's **AccessControl** library. OpenZeppelin's **AccessControl** includes a public `renounceRole()` function that allows any account to renounce a role assigned to itself. While **SingleAdminAccessControl** overrides `renounceRole()` to prevent direct renouncement of the `DEFAULT_ADMIN_ROLE`, there is no additional override or restriction in the current `OverlayerWrap` implementation to prevent renouncement of the `BLACKLISTED_ROLE`.

```
// SingleAdminAccessControl.sol
function renounceRole(
    bytes32 role_,
    address account_
) public virtual override notAdmin(role_) {
    super.renounceRole(role_, account_);
}

modifier notAdmin(bytes32 role_) {
    if (role_ == DEFAULT_ADMIN_ROLE) revert InvalidAdminChange();
    _;
}
```

An account that has been assigned the `BLACKLISTED_ROLE` can call `renounceRole(BLACKLISTED_ROLE, msg.sender)` and independently remove the

restriction, effectively bypassing the blacklist mechanism. Given that, blacklisted users can remove their blacklist status and regain full transfer, mint and reedem functionality, effectively bypassing the administrative control mechanism.

This behavior undermines the intended blacklist enforcement logic. The blacklist is expected to be an administrative control used to restrict malicious, sanctioned, or non-compliant accounts. If users can remove their own blacklist status, the restriction becomes ineffective and cannot be reliably enforced.

Assets:

- contracts/overlayer/OverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/shared/SingleAdminAccessControl.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Consider overriding the `renounceRole()` function in the **OverlayerWrap** contract to prevent accounts from renouncing the `BLACKLISTED_ROLE` themselves.

Alternatively, disable `renounceRole()` entirely if self-removal of roles is not required within the protocol's access control design. This ensures that only authorized administrators can manage blacklist assignments and preserves the integrity of the restriction mechanism.

Resolution: Fixed in `2f2f9d4`, the `renounceRole()` function was overridden to prevent accounts from renouncing the `BLACKLISTED_ROLE` themselves:

```
function renounceRole(bytes32 role_, address account_) public override {
    if (role_ == BLACKLISTED_ROLE)
        revert OverlayerWrapCannotRenounceBlacklist();
```

```
    super.renounceRole(role_, account_);
}
```

Evidences

PoC: Can self renounce BLACKLISTED_ROLE

Reproduce:

PoC step:

1. The **Gatekeeper** assigns the `BLACKLISTED_ROLE` to Bob's address.
2. Bob independently calls `renounceRole()`.
3. As a result, Bob successfully removes the `BLACKLISTED_ROLE` from himself.
4. Bob regains full access to the protocol, effectively bypassing the intended administrative restriction.

PoC code:

```
it("PoC: Can self renounce BLACKLISTED_ROLE", async function () {
    const { overlayerWrap, gatekeeper, alice, bob, admin } =
      await loadFixture(deployFixture);

    const bobAddress = await bob.getAddress();

    const lastTime = await time.latest();
    await overlayerWrap.grantRole(
      ethers.keccak256(ethers.toUtf8Bytes("CONTROLLER_ROLE")),
      gatekeeper.address
    );

    // Sett blacklist activation time
    await overlayerWrap.connect(gatekeeper).setBlackListTime(lastTime + 2);
    await time.increase(3600 * 24 * 15 );

    // Adding bob account to blacklist
    await overlayerWrap.connect(gatekeeper).disableAccount(bobAddress);
    const role = ethers.keccak256(ethers.toUtf8Bytes("BLACKLISTED_ROLE"));
    expect(await overlayerWrap.hasRole(role, bobAddress)).to.be.equal(true);

    // Bob renounce blacklist role independetly and regain full access to the
protocol
    await overlayerWrap.connect(bob).renounceRole(role, bobAddress);
    expect(await overlayerWrap.hasRole(role, bobAddress)).to.be.equal(false);
  });
```

Results:

```
OverlayerWrap
  PoC
    ✔ PoC: Can self renounce BLACKLISTED_ROLE
 1 passing (8s)
```

Files: Blacklist_Renounce_PoC.ts

F-2026-15250 - Hub Chain OverLayer Supply Reduction After OFT Transfers Lead to supply() DoS - High

Description:

The **OverlayerWrap** token supports LayerZero's OFT standard, allowing cross-chain transfers between supported networks (currently: Ethereum <> Base). Under the OFT model, bridging is implemented via burn-on-source / mint-on-destination (and vice versa), which preserves the global supply across chains but changes the `totalSupply()` value on any single chain depending on how much liquidity has migrated.

OverlayerWrap is minted 1:1 upon collateral deposits. The protocol can then supply deposited collateral to Aave via `AaveHandler::supply()` to start generating yield:

```
function supply(
    uint256 amountCollateral_,
    address collateralToken_
) external onlyProtocol nonReentrant {
    //... supply functionality

    // Do not count donations to overlayerWrap: compute how much we have to i
ncrease our supply counters.
    // We cannot exceed the overlayerWrap supply.
    uint256 owTotalSupp = IOverlayerWrap(overlayerWrap).totalSupply();
    if (owTotalSupp < DECIMALS_DIFF_AMOUNT)
        revert AaveHandlerOverlayerWrapTotalSupplyTooLow();
    // Total supply cannot be less than total supplied collateral as ow token
is not burnable
    uint256 normalizedSupply = owTotalSupp / DECIMALS_DIFF_AMOUNT;
    uint256 differenceCollateral = normalizedSupply -
        totalSuppliedCollateral;
    uint256 minIncrease = Math.min(amountCollateral_, differenceCollateral);
    totalSuppliedCollateral += minIncrease;

    emit AaveSupply(minIncrease);
}
```

The accounting logic assumes that the hub-chain `OverlayerWrap::totalSupply()` is monotonically non-decreasing (or at least never falls below the tracked `totalSuppliedCollateral`). This assumption is explicitly reflected in the inline comment ("ow token is not burnable"). However, this assumption is invalid for an OFT token: **cross-chain transfers burn tokens on the source chain**, which can reduce `totalSupply()` on the hub chain.

As a result, the `normalizedSupply` can become **less than** `totalSuppliedCollateral`. In Solidity ≥0.8, the subtraction `differenceCollateral = normalizedSupply - totalSuppliedCollateral;` will underflow and revert. Once this condition is reached, `supply()` becomes permanently unusable (until supply returns), creating a denial of service on the Aave supply path.

If `totalSuppliedCollateral > normalizedSupply` due to hub-chain supply decreasing after OFT transfers, `supply()` will revert due to arithmetic underflow. This prevents newly deposited (or previously unsupplied) collateral from being supplied to Aave, breaking the protocol's yield-generation workflow.

This creates a protocol-level Denial of Service condition for the yield-supply path:

- `supplyToBacking()/supply()` can become non-functional after sufficient cross-chain transfers (if some inactive or inaccessible wallets did not transfer token back to source chain – it becomes permanent)
- Newly deposited collateral (or collateral not yet supplied) may become impossible to supply to Aave, preventing yield generation.
- Collateral remaining idle instead of earning yield.
- The protocol can become operationally inefficient and fail to deliver its intended yield-backed design, degrading user trust and potentially impacting economic assumptions across the system.

Assets:

- contracts/overlayerbacking/AaveHandler.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Exploitability: Independent

Complexity: Simple

Severity: High

Recommendations

Remediation: Revise the `AaveHandler::supply()` delta calculation for updating `totalSuppliedCollateral` so it is resilient to OFT burn/mint behavior and cannot underflow due to cross-chain supply changes.

Resolution: Fixed in `f55efce`, the bridged supply is now stored separately in the `totalBridgedOut` variable. The `supply()` function now checks not only `totalSupply()`, but also the bridged amount, to ensure the total supplied amount remains consistent:

```
uint256 owTotalSupp = IOverlayerWrap(overlayerWrap).totalSupply() +
   IOverlayerWrap(overlayerWrap).totalBridgedOut();
```

Evidences

supply() DoS Due to Cross-chain Transfer Execution

Reproduce:

PoC steps:

1. User creates a new order and mints 100 `OverlayerWrap` tokens.
2. User supplies the deposited collateral to Aave via the `supplyToBacking()` function.
    1. At this point, `totalSuppliedCollateral` is updated to reflect the supplied amount (normalized to 100).
3. Simulate a cross-chain transfer by invoking the internal `_debit()` function (which is executed during a LayerZero `send()` call).
4. As a result of the OFT mechanism, 50 tokens are burned on the source chain.
5. The local `totalSupply()` on the hub chain is now 50.
6. User creates another order and mints 40 additional `OverlayerWrap` tokens.
    1. New local `totalSupply()` = 90.
    2. However, `totalSuppliedCollateral` is still 100.
7. User attempts to call `supplyToBacking()` again.
    1. During execution, the calculation `normalizedSupply - totalSuppliedCollateral` results in an arithmetic underflow (90 − 100).
    2. The transaction reverts due to an overflow/underflow error, effectively causing a DoS of the `supply()` functionality.

PoC code:

```
it("PoC: supplyToBacking()) reverts after simulates crosschain transfer - bur
n (LZ)", async function () {
  const { admin, overlayerWrap, overlayerWrapBacking, usdt } = await loadFixt
ure(deployFixtureWithMockOW);

  const owAddress = await overlayerWrap.getAddress();
  const dec = await usdt.decimals();
```

```
  await usdt.connect(admin).approve(owAddress, ethers.MaxUint256);
  // Minting new order - totalSupply 100
  const mintAmount = ethers.parseUnits("100", dec);
  const order = {
    benefactor: admin.address,
    beneficiary: admin.address,
    collateral: await usdt.getAddress(),
    collateralAmount: mintAmount,
    overlayerWrapAmount: ethers.parseEther("100"),
  };
  await overlayerWrap.connect(admin).mint(order);
  // Supply to backing - totalSuppliedCollateral = totalSupply
  await overlayerWrap.connect(admin).supplyToBacking(mintAmount, 0n);
  expect(await overlayerWrapBacking.totalSuppliedCollateral()).to.equal(mintA
mount);
  expect(await overlayerWrap.totalSupply()).to.equal(ethers.parseEther("100")
);

  const burnAmount = ethers.parseEther("50");
  // Simulate layer zero send by calling internal _debit() function from OFT
  await overlayerWrap.testDebit(admin.address, burnAmount, burnAmount, 0);

  expect(await overlayerWrap.totalSupply()).to.equal(ethers.parseEther("50"))
;
  expect(await overlayerWrapBacking.totalSuppliedCollateral()).to.equal(mintA
mount);

  // second mint order 40 token -> totalSupply (90) < totalSuppliedCollateral
  const mintAmountNew = ethers.parseUnits("40", dec);
  const orderNew = {
    benefactor: admin.address,
    beneficiary: admin.address,
    collateral: await usdt.getAddress(),
    collateralAmount: mintAmountNew,
    overlayerWrapAmount: ethers.parseEther("40"),
  };
  await overlayerWrap.connect(admin).mint(orderNew);
  expect(await overlayerWrap.totalSupply()).to.equal(ethers.parseEther("90"))
;
  expect(await overlayerWrapBacking.totalSuppliedCollateral()).to.equal(ether
s.parseEther("100"));

  // New Collatural cannot be supplied due to overflow error
  await expect(overlayerWrap.connect(admin).supplyToBacking(mintAmountNew, 0)
).to.be.reverted;
});
```

Results:

```
  AaveHandler supply() - Cross-chain totalSupply underflow PoC
    ✔ PoC: supplyToBacking()) reverts after simulates crosschain transfer - b
urn (LZ)
  1 passing (10s)
```

Files: Supply_CrossChainUnderflow_PoC.ts

F-2026-15229 - Cooldown Timer Reset on Repeated Calls Leads to Extended Staking on Previously Queued Assets - Medium

Description:

The cooldown functions manage a per-user struct with a timestamp and an asset accumulator. On each call, the timestamp is overwritten with a fresh deadline while the asset amount is incremented:

```
cooldowns[msg.sender].cooldownEnd = uint104(block.timestamp) + cooldownDurati
on;
cooldowns[msg.sender].underlyingAmount += uint152(assets_);
```

A user who calls the cooldown function twice — for example, once to queue 100 OverlayerWrap and again 30 days later to queue 50 more — will have their deadline reset to a new countdown from the second call. The 100 OverlayerWrap from the first call must wait a new full `cooldownDuration` (90 days), effectively losing the 30 days already accrued under the previous cooldown. During the entire cooldown period, assets sit in the Silo contract, which is a simple transfer-only holder with no yield generation.

There is no cancel function. There is no partial unstake — the claim function releases the full queued amount or nothing:

```
if (block.timestamp >= userCooldown.cooldownEnd || cooldownDuration == 0) {
    userCooldown.cooldownEnd = 0;
    userCooldown.underlyingAmount = 0;
    SILO.withdraw(receiver_, assets);
} else {
    revert IStakedOverlayerWrapCooldownInvalidCooldown();
}
```

A user who discovers the timer reset has no mechanism to cancel or preserve the original cooldown: the assets remain locked earning zero yield until the new cooldown expires. No third-party griefing vector exists — only the user themselves can trigger their own cooldown. The impact is self-inflicted, but the absence of any documentation warning or revert-on-existing-cooldown guard makes this an unguarded state transition that silently penalizes repeat callers. Yield loss is real and proportional to the cooldown duration (up to 90 days of foregone staking rewards).

Assets:

- contracts/overlayer/StakedOverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 4/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: The following remediation options address the timer-reset behavior at different levels of complexity:

- Reject cooldown calls when an active cooldown already exists as a short term solution.
- Introduce a `cancelCooldown()` function that returns the escrowed assets from the `OverlayerWrapSilo` back to the staking vault, crediting the staker with equivalent shares. This provides a recovery path for stakers who trigger the timer reset inadvertently.
- Implement per-cooldown tracking using a mapping of cooldown IDs or an array of `UserCooldown` structs, allowing each cooldown batch to maintain an independent timer. This enables additive cooldowns without interfering with previously queued assets.

Resolution: Fixed in `d07a0a1`, the cooldown functionality was removed. The **StakedOverlayerWrap** vault now operates solely under the original ERC-4626 flow. This allows users to withdraw their assets and rewards immediately, without any time restriction.

F-2026-15242 - Default Admin Can Assign Blacklisted Role Without Enforcing Blacklist Activation Constraints - Medium

Description:

The **OverlayerWrap** token implements a blacklisting mechanism by assigning the `BLACKLISTED_ROLE` using OpenZeppelin's **AccessControl** library. Accounts that are granted the blacklisted role are restricted from performing minting, redeeming, and transfer operations. The contract introduces the `blacklistActivationTime` variable, which defines when the blacklist functionality becomes active. Until the activation time is reached, blacklist operations are not intended to be applied to any account.

The `disableAccount()` function allows accounts to be added to the blacklist by the `CONTROLLER_ROLE`:

```
function disableAccount(
    address account_
) external blacklistAllowed onlyRole(CONTROLLER_ROLE) {
    _grantRole(BLACKLISTED_ROLE, account_);
    emit DisableAccount(account_);
}

function enableAccount(
    address account_
) external blacklistAllowed onlyRole(CONTROLLER_ROLE) {
    _revokeRole(BLACKLISTED_ROLE, account_);
    emit EnableAccount(account_);
}
```

The `blacklistAllowed` modifier enforces the activation timing restriction:

```
modifier blacklistAllowed() {
    if (
        blacklistActivationTime == 0 ||
        blacklistActivationTime + BLACKLIST_ACTIVATION_TIME >
        block.timestamp
    ) {
        revert OverlayerWrapBlacklistNotActive();
    }
    _;
}
```

However, the contract inherits OpenZeppelin's **AccessControl**, which exposes the public `grantRole()` and `revokeRole()` functions. The `DEFAULT_ADMIN_ROLE` can therefore call `grantRole(BLACKLISTED_ROLE, account)`

directly. This direct call does **not** enforce the `blacklistAllowed` modifier and therefore does not apply the intended activation-time restriction.. As a result, the default admin can blacklist (or remove blacklist via `revokeRole()`) accounts at any time, regardless of the configured activation logic.

The same pattern is applied in the **StakedOverlayerWrapCore** contract for blacklisting functionality.

The contract exposes `addToBlacklist()` and `removeFromBlacklist()` functions, which are restricted to accounts holding the `CONTROLLER_ROLE`. These functions are additionally protected by the `blacklistAllowed` modifier, which ensures that the blacklist mechanism is active. The `blacklistAllowed` modifier verifies that `blacklistActivationTime > 0` and that redistribution is not enabled, thereby enforcing that blacklist operations can only be performed when the blacklist functionality is properly activated.

```
function addToBlacklist(
    address target_,
    bool isFullBlacklisting_
) external blacklistAllowed onlyRole(CONTROLLER_ROLE) notOwner(target_) {
    bytes32 role = isFullBlacklisting_
        ? WHOLE_RESTRICTED_ROLE
        : STAKE_RESTRICTED_ROLE;
    _grantRole(role, target_);
}

function removeFromBlacklist(
    address target_,
    bool isFullBlacklisting_
) external blacklistAllowed onlyRole(CONTROLLER_ROLE) {
    bytes32 role = isFullBlacklisting_
        ? WHOLE_RESTRICTED_ROLE
        : STAKE_RESTRICTED_ROLE;
    _revokeRole(role, target_);
}

modifier blacklistAllowed() {
    if (blacklistActivationTime == 0) {
        revert StakedOverlayerWrapCannotBlacklist();
    }
    if (
        blacklistActivationTime + BLACKLIST_ACTIVATION_TIME >
        block.timestamp ||
        redistributionActivationTime > 0
    ) {
        revert StakedOverlayerWrapCannotBlacklist();
    }
```

```
    _;
}
```

The default admin may directly invoke `grantRole()` or `revokeRole()` for `STAKE_RESTRICTED_ROLE` or `WHOLE_RESTRICTED_ROLE`. Consequently, the admin can assign blacklist-related roles at any time, regardless of whether `blacklistActivationTime` has been reached, thereby circumventing the intended activation restrictions.

This can lead to:

- The intended blacklist activation delay restriction logic is not consistently enforced.
- A mismatch between declared behavior and actual enforceable behavior, potentially impacting user trust and protocol transparency.

Assets:

- contracts/overlayer/OverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 5/5

Exploitability: Dependent

Complexity: Simple

Severity: Medium

Recommendations

Remediation: To ensure consistent enforcement of blacklist activation rules:

- Override `grantRole()` and `revokeRole()` in the **OverlayerWrap** and **StakedOverlayerWrapCore** contract to prevent direct assignment or removal of `BLACKLISTED_ROLE` / `WHOLE_RESTRICTED_ROLE` / `STAKE_RESTRICTED_ROLE` via the public **AccessControl** interface by the `DEFAULT_ADMIN_ROLE`.
- Enforce the `blacklistAllowed` restriction within any logic path that assigns `BLACKLISTED_ROLE` / `WHOLE_RESTRICTED_ROLE` / `STAKE_RESTRICTED_ROLE`.

This ensures that all blacklisting actions follow the intended activation constraints and preserves the integrity of the protocol's expected behavior.

Resolution:

Fixed in `e9f1389`, `grantRole()` and `revokeRole()` functions were overridden in the **OverlayerWrap** and **StakedOverlayerWrapCore** contract to prevent direct assignment or the custom role that reguire `blacklistAllowed` validation:

```
// OverlayerWrap.sol
function grantRole(bytes32 role_, address account_) public override {
    if (role_ == BLACKLISTED_ROLE)
        revert OverlayerWrapCannotDirectlyAssignBlacklist();
    super.grantRole(role_, account_);
}

function revokeRole(bytes32 role_, address account_) public override {
    if (role_ == BLACKLISTED_ROLE)
        revert OverlayerWrapCannotDirectlyAssignBlacklist();
    super.revokeRole(role_, account_);
}

// StakedOverlayerWrapCore.sol
function grantRole(
    bytes32 role_,
    address account_
) public virtual override {
    if (role_ == STAKE_RESTRICTED_ROLE || role_ == WHOLE_RESTRICTED_ROLE)
        revert StakedOverlayerWrapCannotDirectlyAssignBlacklist();
    super.grantRole(role_, account_);
}

function revokeRole(
    bytes32 role_,
    address account_
) public virtual override {
    if (role_ == STAKE_RESTRICTED_ROLE || role_ == WHOLE_RESTRICTED_ROLE)
        revert StakedOverlayerWrapCannotDirectlyAssignBlacklist();
    super.revokeRole(role_, account_);
}
```

F-2026-15236 - Instant Change Of Backing Contract Address Can Disable All Vault Operations - Low

Description:

The admin can instantly change the backing contract address without a timelock:

```
function setOverlayerWrapBacking(address backing_) external onlyRole(DEFAULT_
ADMIN_ROLE) {
    overlayerWrapBacking = backing_;
    emit OverlayerWrapBackingSet(backing_);
}
```

This address is called in every vault entry point — deposit, withdraw, redeem, unstake, and both cooldown functions — to trigger a compound call before the operation. The compound call is not wrapped in a try-catch:

```
if (overlayerWrapBacking != address(0)) {
    IOverlayerWrapBacking(overlayerWrapBacking).compound(withdrawAaveDuringCo
mpound);
}
```

If the admin sets this to a reverting contract, all vault operations are denied of service. This is inconsistent with other protocol parameters that use timelocks: Aave migration uses a 10-day proposal-acceptance pattern, collateral spender changes use a 10-day pattern, and the per-block redeem limit uses a 15-day delay. The admin can also fix the issue instantly by setting the value to the zero address, which disables the compound call entirely. The NatSpec confirms that zero address is an intentional disable mechanism.

Assets:

- contracts/overlayer/StakedOverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation: Introduce a propose-accept pattern with a timelock consistent with other protocol parameters (e.g., 10 days).

Resolution: Fixed in `6fb9a40`, propose-accept pattern was implemented with a 15 days timelock change delay:

```
function proposeOverlayerWrapBacking(
    address backing_
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (backing_ == address(0))
        revert StakedOverlayerWrapInvalidZeroAddress();
    proposedOverlayerWrapBacking = backing_;
    proposedOverlayerWrapBackingTime = block.timestamp;
    emit ProposedOverlayerWrapBacking(backing_, block.timestamp);
}

function executeOverlayerWrapBackingChange()
    external
    onlyRole(DEFAULT_ADMIN_ROLE)
{
    if (proposedOverlayerWrapBacking == address(0)) {
        revert StakedOverlayerWrapInvalidZeroAddress();
    }
    if (overlayerWrapBacking != address(0)) {
        if (
            block.timestamp <
            proposedOverlayerWrapBackingTime + BACKING_CHANGE_DELAY
        ) {
            revert StakedOverlayerWrapBackingChangeDelayNotRespected();
        }
    }

    address newBacking = proposedOverlayerWrapBacking;

    proposedOverlayerWrapBacking = address(0);
    proposedOverlayerWrapBackingTime = 0;

    overlayerWrapBacking = newBacking;
    emit OverlayerWrapBackingSet(newBacking);
}
```

F-2026-15237 - Emergency Disable Function Only Halts Minting Despite NatSpec Claiming Both Mint And Redeem Are Disabled - Low

Description:

The emergency disable function is documented as halting both minting and redemptions, but its implementation only disables minting:

```
/// @notice Disables the mint and redeem
function disableMint() external onlyRole(GATEKEEPER_ROLE) {
  _setMaxMintPerBlock(0);
}
```

The gatekeeper role has no mechanism to disable or reduce redemptions. The per-block redeem limit can only be changed by the admin through a 15-day timelocked proposal flow. The setter function explicitly prevents the value from being set to zero or below a minimum floor:

```
function _setMaxRedeemPerBlock(uint256 maxRedeemPerBlock_) internal {
  if (maxRedeemPerBlock_ == 0 || maxRedeemPerBlock_ < minValmaxRedeemPerBlock
) {
    revert OverlayerWrapCoreInvalidMaxRedeemAmount();
  }
  // ...
}
```

This means redemptions can never be fully halted — not even by the admin. The minimum redemption throughput is permanently bounded by an immutable floor value set at construction. The pause function only gates the supply-to-backing call, not redemptions. In an emergency scenario requiring a complete protocol freeze, the current design does not provide a path to halt both minting and redeeming.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Exploitability: Semi-Dependent

Complexity: Simple

Severity: Low

Recommendations

Remediation:

- Correct the NatSpec to accurately reflect the function behavior: "Disables minting by setting the per-block mint limit to zero."
- Evaluate whether the inability to fully halt redemptions is an intended invariant or an oversight, and document the decision then consider adding a gatekeeper-callable function that can reduce the redeem limit to its minimum floor.

Resolution: Fixed in `4f072f6`, the NatSpec was updated to correctly reflect current contract behavior:

```
/// @dev Can be disabled by the gatekeeper via disableMint()
/// @param order_ A struct containing the mint order
function mint(...) {
    ...
}

/// @dev Cannot be disabled or paused
/// @param order_ A struct containing the redeem order
function redeem(...) {
    ...
}
```

F-2026-15233 - Floating Pragma Across All Contracts - Info

Description:

All in-scope Solidity files use a floating pragma:

```
pragma solidity ^0.8.20;
```

This permits compilation with any Solidity version from 0.8.20 up to (but not including) 0.9.0. Different compiler versions can produce different bytecode, apply different optimizations, and introduce subtle behavioral differences. The deployed bytecode depends on which compiler version the build environment happens to use, reducing reproducibility and complicating on-chain verification.

In a production deployment context, pinning the pragma to a specific version ensures that the audited source corresponds exactly to the deployed bytecode and that bytecode verification on block explorers produces deterministic matches.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayerbacking/AaveHandler.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/StakedOverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/StakedOverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/OverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/CollateralSpenderManager.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/shared/SingleAdminAccessControl.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayerbacking/OverlayerBacking.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/OverlayerWrapCollateral.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/interfaces/IOverlayerWrapDefs.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/OverlayerWrapSilo.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/types/OverlayerWrapCoreTypes.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Pin the pragma to a specific compiler version across all contracts (e.g., `pragma solidity 0.8.28`).
- Alternatively, enforce the compiler version in the Hardhat configuration while keeping the floating pragma. This provides reproducibility at the build level without modifying source files.

Resolution: The pragma version is enforced by the Hardhat configuration to build contract under the `0.8.20` solidity version:

```
{
  version: "0.8.20",
  settings: {
    optimizer: {
      enabled: true,
      runs: 300,
    },
    // viaIR: true
  },
},
```

F-2026-15239 - No Cancellation Mechanism For Pending Per-Block Redeem Limit Proposals - Info

Description:

The per-block redeem limit uses a 15-day timelocked proposal flow. The admin submits a proposal, which stores the new value and a timestamp:

```
function proposeMaxRedeemPerBlock(uint256 newMaxRedeemPerBlock_) external onl
yRole(DEFAULT_ADMIN_ROLE) {
    if (newMaxRedeemPerBlock_ == 0 || newMaxRedeemPerBlock_ < minValmaxRedeem
PerBlock) {
        revert OverlayerWrapCoreInvalidMaxRedeemAmount();
    }
    proposedRedeemChangeTime = block.timestamp;
    proposedMaxRedeemPerBlock = newMaxRedeemPerBlock_;
    emit ProposedMaxRedeemPerBlock(newMaxRedeemPerBlock_, block.timestamp);
}
```

After the 15-day delay, the admin can execute the change:

```
function executeMaxRedeemPerBlockChange() external onlyRole(DEFAULT_ADMIN_ROL
E) {
    if (proposedRedeemChangeTime == 0) {
        revert OverlayerWrapCoreInvalidMaxRedeemAmount();
    }
    if (block.timestamp < proposedRedeemChangeTime + REDEEM_CHANGE_DELAY) {
        revert OverlayerWrapCoreDelayNotRespected();
    }

    uint256 newValue = proposedMaxRedeemPerBlock;

    // reset proposal state
    proposedRedeemChangeTime = 0;
    proposedMaxRedeemPerBlock = 0;

    _setMaxRedeemPerBlock(newValue);
}
```

Once a proposal is submitted, there is no dedicated cancel function to discard it. The proposed value and timestamp persist until overwritten by a new proposal or consumed by execution.

Stale proposals cannot auto-execute — the execution function requires the 15-day delay to have elapsed and can only be called by the admin. The admin can effectively cancel by submitting a new proposal with the current

value, overwriting the pending one. This is an operational inconvenience rather than a security concern.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Exploitability: Dependent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

- Consider adding a dedicated cancel function that resets the proposal state to zero, gated by the admin role.
- Alternatively, accept the current design and document the overwrite-to-cancel pattern.

Resolution: Fixed in `8fc165e`, the dedicated cancel function that resets the proposal state to zero was introduced:

```
function cancelProposedMaxRedeemPerBlock()
  external
  onlyRole(DEFAULT_ADMIN_ROLE)
{
  proposedRedeemChangeTime = 0;
  proposedMaxRedeemPerBlock = 0;
}
```

F-2026-15244 - Lack of Event Emitting for Key State Changes - Info

Description:

Several functions within the **Overlayer** protocol perform meaningful state changes and token movements without emitting events, which hinders off-chain observability:

- `OverlayerWrap::setBlackListTime()`
- `OverlayerWrapCore::whitelistMaxRedeemPerBlockUser()`
- `StakedOverlayerWrap::unstake()`
- `StakedOverlayerWrapCore::rescue()`
- `AaveHandler::proposeNewAave()`
- `AaveHandler::proposeNewOvaDispatcherAllocation()`
- `AaveHandler::approveAave()`
- `AaveHandler::approveStakingOverlayerWrap()`
- `AaveHandler::approveOverlayerWrap()`
- `AaveHandler::adminWithdraw()`

The absence of an event makes it difficult to track admin-specific actions off-chain, which reduces transparency and complicates monitoring and auditing. It is recommended to emit events for this operation to ensure proper visibility, support off-chain indexing, and facilitate debugging, especially for critical administrative and user-facing fund transfers.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayerbacking/AaveHandler.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/StakedOverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/StakedOverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]
- contracts/overlayer/OverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider emitting events within the mentioned affected functions to enhance transparency and ensure reliable off-chain tracking of key state changes. Emitting events not only improves monitoring of critical storage variable updates and key operation execution but also supports indexing services, facilitates auditing, and provides better visibility into administrative actions.

Resolution: Fixed in `1d3afad`, event emitting was implemented in all mentioned functions.

F-2026-15245 - Inconsistency Between Documentation and Implementation in removeCollateralManagerRole() - Info

Description:

The **CollateralSpenderManager** implements `COLLATERAL_MANAGER_ROLE`, which is responsible for proposing the approved collateral spender account `approvedCollateralSpender`.

The `removeCollateralManagerRole()` function in the **OverlayerWrapCore** is intended to revoke the `COLLATERAL_MANAGER_ROLE` from a specified account. According to the inline documentation, the `GATEKEEPER_ROLE` is expected to have the authority to remove this role.

```
/// @notice Removes the collateral manager role from an account, this can ONL
Y be executed by the gatekeeper role
/// @param collateralManager_ The address to remove the COLLATERAL_MANAGER_RO
LE role from
function removeCollateralManagerRole(
    address collateralManager_
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    _revokeRole(COLLATERAL_MANAGER_ROLE, collateralManager_);
}
```

However, in the current implementation, the function is restricted to `DEFAULT_ADMIN_ROLE`, not `GATEKEEPER_ROLE`, which contradicts the documented behavior.

This leads to misalignment between documented and actual access control behavior which introduce operational confusion regarding which role is responsible for managing collateral manager permissions.

Additionally, role revocation can already be performed via the public `revokeRole()` function inherited from OpenZeppelin's **AccessControl**, allowing `DEFAULT_ADMIN_ROLE` to revoke `COLLATERAL_MANAGER_ROLE` directly, bypassing the `removeCollateralManagerRole()` function entirely.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate:	1/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Clarify and align the intended access control model:

- If `GATEKEEPER_ROLE` is intended to be the only role authorized to remove `COLLATERAL_MANAGER_ROLE`, update the function access modifier accordingly and consider overriding `revokeRole()` to prevent `DEFAULT_ADMIN_ROLE` from bypassing the intended restriction.
- If `DEFAULT_ADMIN_ROLE` is intended to retain ultimate authority over role revocation, update the inline documentation to reflect the actual implementation and consider removing the `removeCollateralManagerRole()` function entirely, as `revokeRole()` already provides this functionality.

Ensuring consistency between documentation and implementation will improve clarity, maintainability, and transparency.

Resolution: Fixed in `360be17`, the `removeCollateralManagerRole()` function was removed.

F-2026-15246 - Redundant Redeem Rate-Limit Check Implemented Both As Modifier And Inline Logic - Info

Description:

A per-block redeem rate limit is defined as a modifier but never attached to any function:

```
modifier belowMaxRedeemPerBlock(uint256 redeemAmount_) {
    if (redeemedPerBlock[block.number] + redeemAmount_ > maxRedeemPerBlock
        && !maxRedeemWhitelist[msg.sender]) {
        revert OverlayerWrapCoreMaxRedeemPerBlockExceeded();
    }
    _;
}
```

Instead, the redeem management function contains an identical inline check that uses the precision-adjusted burn amount rather than the raw order amount:

```
// Inside _managerRedeem():
if (redeemedPerBlock[block.number] + checkedBurnAmount > maxRedeemPerBlock
    && !maxRedeemWhitelist[msg.sender]) {
    revert OverlayerWrapCoreMaxRedeemPerBlockExceeded();
}
```

The inline version is the correct implementation because it operates on the post-adjustment value. The modifier remains as unused dead code.

Assets:

- contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Remove the unused modifier to reduce code size and eliminate confusion about which rate-limit path is. Or, refactor the redeem function to use the modifier if the pre-adjustment check is acceptable.

Resolution: Fixed in `586c82a`, the unused modifier was removed.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

hknio/severity-formula

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
Repository	https://github.com/Overlayerfi/contracts/tree/main
Initial Commit	471a134a41218fe145dd10be570d8a6755b25b7c
Final Commit	519c9e92fd9d80d11e35e9868130f6334b88d676
Whitepaper	N/A
Requirements	./Readme.md
Technical Requirements	./Readme.md

Asset	Type
contracts/overlayer/CollateralSpenderManager.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/interfaces/IOverlayerWrapDefs.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/OverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/OverlayerWrapCollateral.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/OverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/OverlayerWrapSilo.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/StakedOverlayerWrap.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/StakedOverlayerWrapCore.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayer/types/OverlayerWrapCoreTypes.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayerbacking/AaveHandler.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/overlayerbacking/OverlayerBacking.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract
contracts/shared/SingleAdminAccessControl.sol [https://github.com/Overlayerfi/contracts/tree/main]	Smart Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.